
Robust QR-DQN Implementation for CleanRL

Matasaru Luxin¹ Alami Simo^{1,2}

¹École Polytechnique ²LIX

Abstract

As interest in reinforcement learning expands, there is a growing need for optimization and standardization of research tools. Well known libraries such as Stable Baselines provide reliable implementations of algorithms. However, they favor modularity and completeness of builtin features at the expense of code readability and ease of modification. CleanRL is a community driven effort that sets out to fill this gap. They provide single file implementations written with experimentation and reproducibility in mind. While certain value estimation algorithms such as DQN and C51 have already been added, QR-DQN, a fundamental result in distributional reinforcement learning, is missing. In this work we present the implementation of a reliable version of QR-DQN. We detail the design choices made and link them back to the theory in Dabney et al. (2017). Finally we show our results on the Classic Control and Atari2600 tasks, proving our implementation can predict correct distributions and perform on par with that of Dabney et al. (2017). Given this, we have submitted our code to CleanRL.

1. Introduction

Reinforcement learning (RL) aims to solve tasks where an algorithm or model, known as agent, can improve itself by interacting with a system or world, known as environment. At each discrete timestep, the agent receives an observation from the environment. It then uses that information to take an action changing the state of the environment for the next timestep. At that point, the agent receives a new observation and possibly a reward.

The goal of the problem is to maximize the sum of rewards discounted across time. One intuitive approach is to model the expectation of this sum given a state and an action, also known as the Q-function. One can then greedily choose the action which maximizes the Q-function. Mnih et al. (2015) show that deep neural networks can be a good tool for this task with the right training.

Bellemare et al. (2017) argue in favor of modeling the distribution of the sum of discounted rewards. They prove the usefulness of this approach with the C51 algorithm which outperforms Mnih et al. (2015)’s DQN. With QR-DQN, Dabney et al. (2017) propose a more robust parametrization for which they prove better theoretical guarantees than C51. This is achieved by modeling cumulative distribution functions instead of the probability laws.

Since these algorithms involve Bootstrapping through the Bellman equation, training is inherently unstable. Thus, recreating results becomes non trivial without a good understanding of the specific setup and many tricks used to ensure stability. In this work we give a complete view of everything it takes to recreate the results of Dabney et al. (2017). Along the way we highlight and motivate the QR-DQN algorithm, environment setup, observation preprocessing, the necessity for a replay buffer and different training

and target models, as well the complete set of hyperparameters.

2. Background

2.1. CleanRL

In this work we present the implementation of the Quantile Regression Deep Q-Learning algorithm for CleanRL. The latter is an open source library providing reliable single file implementations of many popular reinforcement learning algorithms. Compared to bigger libraries such as Stable Baselines, CleanRL aims to keep its codebase minimalistic, trading efficiency and modularity for code clarity and ease of experimentation. Thus the library appeals more to RL research rather than robust model deployment. Further details can be found on the project’s [main page](#).

2.2. Reinforcement Learning

¹ Formally, agent-environment interactions are modeled using a Markov decision process (MDP) $(\chi, \mathcal{A}, R, P, \gamma)$ with χ and $\mathcal{A}$ the state and action spaces, R the reward random variable, $P(x'|x, a)$ the probability of transitioning from state x to state x' after taking action a , and $\gamma \in [0, 1)$ the discount factor applied to future rewards. We say that the agent follows a policy $\pi(\cdot|x)$, a mapping from $x \in \chi$ to a probability distribution over $\mathcal{A}$.

The goal of an RL agent is to maximize the total discounted return, G .

$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

Where r_t is the reward received at time t .

We consider the return of a policy π , from taking action $a \in \mathcal{A}$ while in state $x \in \chi$

$$Z(x, a) = \sum_{t=0}^{\infty} \gamma^t R_t(x_t, a_t)$$

$$x_t \sim P(\cdot|x_{t-1}, a_{t-1}), \quad a_t \sim \pi(\cdot|x_t), \quad x_0 = x \quad a_0 = a$$

Where $R_t(x_t, a_t)$ is the random variable representing the reward at moment t . Thus G is just a realization of the random variable Z . Depending on how this maximization is achieved, we distinguish two major types of RL algorithms: value iteration and policy iteration.

Value Iteration

The first aims to predict the the Q-function $Q(x, a) = \mathbb{E}[Z(x, a)]$. To do this, it makes use of the Bellman equation.

The Bellman equation (Bellman, 1958) is a recurrence relation between the Q-values of consecutive state-action pairs. For a given policy π we have:

$$Q(x, a) = \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_{P, \pi}[Q(x', a')] \\ x' \sim P(\cdot | x, a), \quad a' \sim \pi(\cdot | x')$$

We define the optimal action-value function $Q^*(x, a)$ as the maximum expected return possible by following any policy. Looking at the Bellman equation, we deduce that a policy achieving $Q^*(x, a)$ will have to pick the next action a' which leads to the best $Q(x', a')$. Otherwise it would be strictly smaller. This is described by the optimality equation

$$Q^*(x, a) = \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_P \left[\max_{a' \in \mathcal{A}} Q^*(x', a') \right]$$

The right hand side can be considered a function in $Q^*(x, a)$, that is, $\mathcal{T}$, the optimality operator. Q^* is thus a fixed point of $\mathcal{T}$, and it is unique as the operator is a contraction.

The Bellman operator $\mathcal{T}^\pi$ can also be defined from the Bellman equation.

$$\mathcal{T}^\pi Q(x, a) := \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_{P, \pi}[Q(x', a')]$$

We are concerned with finding the fixed point of $\mathcal{T}$ or $\mathcal{T}^\pi$. Value iteration performs this by repeated application of the operators, but other methods can also be used.

From there, an agent can simply follow a greedy policy, picking the action which maximizes $Q(x, a)$ at each step.

Policy Iteration

While value iteration iterates over value functions, policy iteration iterates over policies themselves seeking to improve them at each step. Starting with a suboptimal policy π_0 , the first step is the so called policy evaluation, where the value of each state is estimated given the current policy and the evaluations of its neighbor states. Formally, for all $x \in \mathcal{X}$:

$$V(x) = \sum_{a \in \mathcal{A}} \pi(a|x) Q(a, x) \\ Q(x, a) = \sum_{x' \in \mathcal{X}} P(x'|x, a) (R(x, a, x') + \gamma V(x'))$$

Where $R(x, a, x')$ is the random reward received by taking action a in state x and arriving in state x' . It relates to $R(x, a)$ by $R(x, a) = \mathbb{E}_{x' \in \mathcal{X}} R(x, a, x')$.

The next step is policy improvement. For each state we look for an action yielding a better Q-value than the one expected from following the current policy. If this exists, we update the policy to follow the better action in the given state. Formally, for every state x if there exists $a \in \mathcal{A}$ such that $Q(x, a) > Q(x, \pi(x))$ then we perform the update $\pi(x) \leftarrow a$.

The repeated application of these two steps forms the basis of policy iteration.

Q-Learning

Methods such as Deep Q-Learning (DQN) (Mnih et al., 2015) and its distributional successors C51 (Bellemare et al., 2017) or QR-DQN (Dabney et al., 2017) also aim to find the fixed point of the Bellman operator. However, instead of iterating $\mathcal{T}^\pi$, they opt for non linear function approximation.

DQN uses deep neural networks to estimate the Q-function. The training loss is

taken as the MSE between the model’s prediction and its application to the Bellman operator. In this case our learning target would be constantly moving as it depends on the trained model. The solution used in practice is to keep separate target and training models which are synchronized at regular intervals. The target model is fixed and its outputs are used only in the Bellman equation.

2.3. Distributional RL

Distributional RL is a subset of the value estimation class of algorithms. Methods in this category model $Q(x, a)$ by first predicting distribution of $Z(x, a)$ given some state and action. The Q value is then recovered as the expected value of this distribution over all sources of randomness intrinsic to the MDP.

Motivation

One may object that the distribution does not play a role in the policy outlined above. We are effectively throwing away most of the information. What is then the justification behind this approach? We identify two main reasons.

Consider a MDP where from the current state x we can choose between two actions a_1 and a_2 both leading to a final state. The first action may yield a reward of 1 or 3 with equal probability, while the second will do the same for rewards 0 and 4. It is easy to see that $Q(x, a_1) = Q(x, a_2) = 2$. However, a_1 is less risky. While classical Q-learning will be indifferent between the two actions, a distributional algorithm could learn the bimodal distributions enabling a policy to distinguish a_1 from a_2 . While the ϵ -greedy policy considered in this work is incapable of this, such a property is desirable in practice.

¹We found Sutton and Barto (2018) to be a great reference for understanding the recurring basic concepts in reinforcement learning.

Even with a policy taking the expectation, the results of Bellemare et al. (2017) and Dabney et al. (2017) empirically show that distributional RL outperforms classical value estimation. A potential explanation is that when $Z(x, a)$ has a multi modal distribution, sample noise during training makes direct Q-value estimates oscillate around the true expectation. In the distributional setting however, the samples will only reinforce the modes within the model, leading to smoother training. For example, consider that $Z(x, a)$ has equal chance of taking the value -10 or 10. Say our current estimate for $Q(x, a)$ is close to 0, and we observe a reward of 10. Since the error is quite big, so will be the gradients and the updated model’s new prediction will deviate significantly from 0. On the flip side, if we model a distribution, the sample from before could slightly increase the density around the value 10, and decrease that around -10. However, the distribution as a whole will keep its structure.

Distributional Bellman Operator

As seen above, the Temporal difference (TD) equation allows us to improve our estimates of the present by incorporating immediate rewards into future predictions.

$$\begin{aligned} \mathcal{T}^\pi Q(x, a) &= \mathbb{E}[R(x, a)] + \gamma \mathbb{E}_{P, \pi}[Q(x', a')] \\ a &\sim \pi(\cdot|x), \quad x' \sim P(\cdot|x, a), \quad a' \sim \pi(\cdot|x') \end{aligned}$$

The Bellman operator can be extended to the distributional case giving the distributional Bellman operator.

$$\mathcal{T}^\pi Z(x, a) \stackrel{D}{=} R(x, a) + \gamma Z(x', a')$$

Where $\stackrel{D}{=}$ represents equality of probability laws.

In the above $x' \sim P(\cdot|x, a)$ and $a' \sim \pi(\cdot|x')$ represent the next state respectively the next action.

From here, we would like to proceed similarly to DQN and minimize some metric between $Z(x, a)$ and $\mathcal{T}^\pi Z(x, a)$. Since we are

working with probability laws, MSE is not an option. A popular choice for computing the similarity in this case is Kullback-Leibler (KL) divergence. Unfortunately this is not a metric on probability distributions (it is not symmetric). This means $\mathcal{T}^\pi$ cannot be a contraction with respect to KL divergence.

There is also the issue that the KL divergence is infinite for distributions with different support. In our case, the Bellman operator shifts and scales the input distribution, so we cannot assume $Z(x, a)$ and $\mathcal{T}^\pi Z(x, a)$ have the same support.

Another good candidate is the Wasserstein metric which we investigate in the following.

The Wasserstein Metric

In order to present some theoretical convergence results in the next sections, we need to first define a notion of distance between probability distributions. In the literature, the p-Wasserstein metric is often used. For any $p \in [0, \infty]$ and probability distributions U, V it is defined as

$$W_p(U, V) = \left(\int_0^1 |F_U^{-1}(\omega) - F_V^{-1}(\omega)|^p d\omega \right)^{\frac{1}{p}}$$

Where F_U^{-1} is the inverse of the CDF of U .

One may notice this is the L^p metric between two inverse CDFs. As such, W_p is a metric on the space of distributions.

Specifying this to distributional RL, we first define the space of all finite moment return distributions.

$$\mathcal{Z} = \{Z : \chi \times \mathcal{A} \rightarrow \mathcal{P}(\mathbb{R}) \mid \mathbb{E}[|Z(x, a)|^p] < \infty, \forall (x, a), p \geq 1\}$$

We can then endow it with the distance given by the maximal form of the Wasserstein metric introduced by Bellemare et al. (2017): For any $Z_1, Z_2 \in \mathcal{Z}$

$$\bar{d}_p(Z_1, Z_2) := \sup_{x, a} W_p(Z_1(x, a), Z_2(x, a))$$

Moreover, the Bellman operator $\mathcal{T}^\pi$ is a contraction in $\bar{d}_p(Z_1, Z_2)$ (Lemma 3 in Bellemare et al. (2017)). This implies that $\mathcal{T}^\pi$ has a unique fixed point, representing the true return distribution we would like to find. One might also be led to believe that we can find it by minimizing the Wasserstein distance between any candidate distribution and its Bellman update.

In other words, starting from $Z \in \mathcal{Z}$ we would like to minimize

$$\bar{d}_p(\mathcal{T}^\pi Z, Z) = \sup_{x, a} W_p(\mathcal{T}^\pi Z(x, a), Z(x, a))$$

In practice, we do not have direct access to $\mathcal{T}^\pi Z$, rather we can sample from it. A natural approach is thus to use stochastic gradient descent. Consider Z_θ the model we are trying to optimize, with parameters $\theta \in \mathbb{R}^n$. Let $\hat{Z}_m = \frac{1}{m} \sum_{i=1}^m \delta_{Y_i}$ with $Y_i \sim \mathcal{T}^\pi Z(x, a)$ an empirical distribution. Using SGD we can then minimize

$$\mathbb{E}_{x, a}[W_p(\hat{Z}_m, Z_\theta(x, a))]$$

Unfortunately, Theorem 1 of Bellemare et al. (2017) shows that this method does not converge to the desired distribution. Formally:

$$\begin{aligned} \arg \min_{\theta} \mathbb{E}_{x, a}[W_p(\hat{Z}_m, Z_\theta(x, a))] &\neq \\ \arg \min_{\theta} W_p(Z(x, a), Z_\theta(x, a)) \end{aligned}$$

Where Z is the unique fixed point of $\mathcal{T}^\pi$.

In the following we will detail how C51 and QR-DQN attempt to overcome this limitation.

2.4. The C51 Algorithm

In Bellemare et al. (2017) the authors propose Z_θ as a discrete distribution whose support is the set of atoms $z_0 < z_1 < \dots < z_{N-1}$ between two values, V_{max} and V_{min} . The atomic probabilities are then given by a parametric model. $\theta : \chi \times \mathcal{A} \rightarrow \mathbb{R}^n$. (Section 4.1 Bellemare et al. (2017))

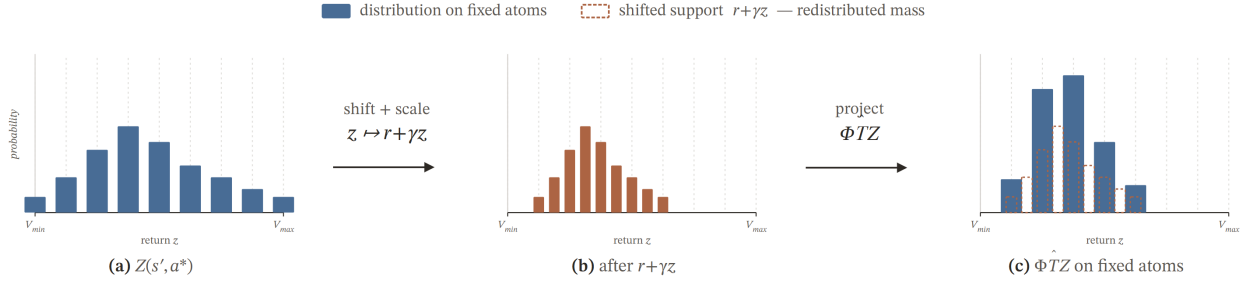


Figure 1: One step of the C51 distributional Bellman update. (a) the next-state value distribution $Z(s', a^*)$ supported on the fixed atoms $z_0 \dots z_{N-1}$. (b) applying the Bellman operator shifts and scales every atom, $z \mapsto r + \gamma z$, so the support contracts toward r and no longer aligns with the grid. (c) the projection $\Phi \hat{T} Z$ redistributes each off-grid atom’s mass onto its two nearest fixed atoms, returning a distribution on the original support (shifted bars shown dashed for reference).

During a TD step, the distribution is shifted by the reward and scaled by the discount factor. The bound values (and thus the atoms) stay fixed. In order to maintain the same support, the authors opt for a projection step. This maps the scaled distribution back onto the original atoms.

As explained in section 4.2 of Bellemare et al. (2017), the combination of TD and this projection defines the projected update operator $\Phi \hat{T}$. The sample loss is then given by the KL divergence between Z_θ and its projected update and it is optimized with SGD.

$$\mathcal{L}_{x,a}(\theta) = D_{KL}(\Phi \hat{T} Z_\theta(x, a) \parallel Z_\theta(x, a))$$

The hat is used to distinguish the fact that temporal difference is performed on a sample from Z_θ .

In spite of the theoretical limitations, C51 surpasses DQN on many tasks and even achieved state of the art performance on multiple games in the Atari 2600 set. This proved the potential of distributional RL to outperform simple value estimation. The generally accepted reasons for this are the ones stated in the Motivation part of Section 2.3.

The lack of theoretical guarantees regarding the convergence of SGD, and the fixed

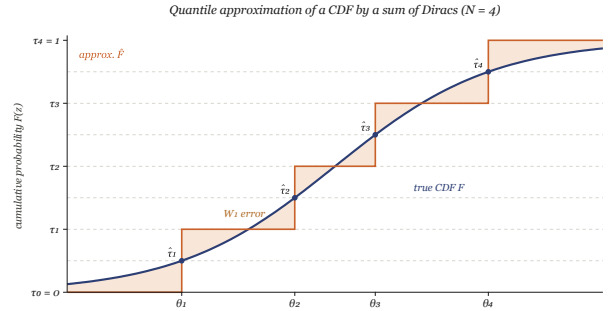


Figure 2: $\theta_i = F^{-1}(\hat{\tau}_i)$ minimize W_1 error between a CDF and its quantile approximation. W_1 error is the area shaded in orange.

support of the modeled distribution imposed by the atoms are the two main drawbacks of C51. The former does not appear to prevent strong empirical performance. The latter is much more limiting as V_{min} , V_{max} and N need to be set before training, thus requiring some prior knowledge of the reward structure. If not done correctly the distribution can be clipped and training will fail silently. This makes C51 poorly suited to problems where the reward scale is unknown or varies significantly.

2.5. The QR-DQN Algorithm

Dabney et al. (2017) sets out to overcome the limitations of C51. Instead of directly modelling the reward distribution, the authors shift their attention to its CDF. QR-DQN

aims to approximate a target distribution via its quantiles (a quantile distribution).

Formally, quantile values $\theta_i(x, a)$ are associated to cumulative probabilities $\tau_1, \dots, \tau_N$ with $\tau_i = \frac{i}{N}$ for $i = 1, \dots, N$. Thus $\theta : \chi \times \mathcal{A} \rightarrow \mathbb{R}^N$ is the parametric model to learn, and the quantile distribution Z_θ maps each state action pair to a probability distribution.

$$Z_\theta(x, a) := \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a)}$$

δ_z denotes a Dirac distribution at $z \in \mathbb{R}$. Moreover, let Z_Q the space of such parametrized distributions.

Lemma 2 of Dabney et al. (2017) shows that given a random variable Y , we would like to approximate its distribution by Z_θ . That is, minimize 1-Wasserstein metric:

$$W_1(Y, Z_\theta) = \sum_{i=1}^N \int_{\tau_{i-1}}^{\tau_i} |F_Y^{-1}(\omega) - \theta_i| d\omega$$

The quantile values θ_i providing the smallest $W_1(Y, Z_\theta)$ are given by the following. (see 2)

$$\theta_i = F_Y^{-1}(\hat{\tau}_i), \quad \hat{\tau}_i = \frac{\tau_i - 1 + \tau_i}{2}$$

Thus, we need to estimate $F_Y^{-1}(\hat{\tau}_i)$ for all $x \in \chi$, $a \in \mathcal{A}$ and $i = 1 \dots N$, where $Y = \mathcal{T}^\pi Z(x, a)$.

Furthermore, Proposition 2 of Dabney et al. (2017) gives that the combined operator of projection onto Z_Q followed by TD is a contraction in $\bar{d}_\infty$ (which we defined in 2.3). In particular, the operator is a contraction in $\bar{d}_1 \leq \bar{d}_\infty$. This implies there exists a unique fixed point $\hat{Z}^\pi$. All that is left is to find it.

Even so, for the same reason as above, SGD cannot be used to optimize the Wasserstein metric as a loss (Proposition 1 Dabney et al. (2017)). The quantile reparametrization provides the solution, as it allows for the use of quantile regression. This is a method for approximating the quantile functions of distributions (Koenker, 2005).

Quantile Huber Loss

The loss function is of particular importance to our goal of implementing QR-DQN. Since the standard quantile regression loss is not smooth around 0, Dabney et al. (2017) opt to merge it with the Huber loss (Huber, 1964). The so called quantile regression Huber loss for a given quantile τ can be expressed as follows:

$$\rho_\tau^\kappa(u) = |\tau - \delta_{u < 0}| \mathcal{L}_\kappa(u) \quad (1)$$

with

$$\mathcal{L}_\kappa(u) = \begin{cases} \frac{1}{2}u^2 & \text{if } |u| \leq \kappa \\ \kappa \left(|u| - \frac{1}{2}\kappa \right) & \text{otherwise} \end{cases}$$

In the above κ is the Huber parameter and is generally set to 1. Setting $\kappa = 0$ is equivalent to using the pure quantile regression loss.

Unlike the Wasserstein metric, the quantile regression loss and its Huber variant provide unbiased gradients such that for a target distribution Z , we can find values of $\{\theta_1, \dots, \theta_N\}$ minimizing $W_1(Z, Z_\theta)$ by optimizing

$$\sum_{i=1}^N \mathbb{E}_{\hat{Z} \sim Z} [\rho_{\hat{\tau}_i}(\hat{Z} - \theta_i)]$$

using stochastic gradient descent.

Since we aim to approximate the fixed point of the projected TD operator, we set $\mathcal{T}Z_\theta(x, a)$ as our target distribution. Next, we can sample state-action pairs (x, a) along with realizations of the resulting reward r and next state x' :

$$x \in \chi, a \sim \pi(\cdot|x), r \sim R(x, a), x' \sim P(\cdot|x, a)$$

$\theta(x, a)$ then defines our current estimate of the quantile distribution $Z_\theta(x, a)$. The ϵ -greedy policy determines the agent's next action a' . Using the Bellman equation we can get the quantile distribution $\mathcal{T}Z_\theta(x, a)$. Being a quantile distribution, we can write

$$\mathcal{T}Z_\theta(x, a) = \frac{1}{N} \sum_{j=1}^N \delta_{\theta'_j(x, a)}$$

The expected value over $\hat{Z} \sim \mathcal{T}Z_\theta(x, a)$ is then approximated by taking the mean of $\rho_{\hat{\tau}_i}(\theta'_j(x, a) - \theta_i(x, a))$ over $j = 1 \dots N$. This summarizes the Quantile Regression Q-Learning algorithm laid out in Algorithm 1 of Dabney et al. (2017).

One detail is that often in reinforcement learning, $\mathcal{T}Z_\theta(x, a)$ is derived from a separate target network which is synchronized with the learner model after a set number of training steps. This is done in order to maintain a stable target (at least briefly) for SGD to move towards. Failing to do this would cause our learning target to shift with the noise in our training steps leading to instability.

Algorithm 1: Quantile Regression Q-Learning (Algorithm 1 Dabney et al. (2017))

Input: $x, a, r, x', \gamma \in [0, 1)$
Data: N, κ
 // Compute distributional
 Bellman target
 $Q(x', a') \leftarrow \sum_j q_j \theta_j(x', a')$;
 $a^* \leftarrow \arg \max_{a'} Q(x, a')$;
 $\mathcal{T}\theta_j \leftarrow r + \gamma \theta_j(x', a^*), \quad \forall j$;
 // Compute quantile regression
 loss (Equation 1)
Output: $\sum_{i=1}^N \mathbb{E}_j [\rho_{\hat{\tau}_i}^\kappa(\mathcal{T}\theta_j - \theta_i(x, a))]$

In their implementation, the authors adapt the DQN architecture (Mnih et al., 2015). The output layer size is changed to $|\mathcal{A}| \times N$ and the RMSProp optimizer is swapped for Adam. The exact hyperparameters used are discussed in the validation section. On the Atari 2600 games, QR-DQN outperforms C51 in most cases, achieving state of the art performance for many of them. However the authors note that RL methods remain limited on a small subset of games.

One major drawback of QR-DQN is its

limitation to tasks with discrete action spaces. Unfortunately many useful applications of RL fall outside this category, requiring a reduction of the action space in order to properly apply the algorithm.

3. Method

3.1. Model Architecture

Similarly to other algorithms in CleanRL, we implement two architectures of QR-DQN. The first is a minimal model which we train on the relatively simple Classic Control tasks from Gymnasium. This is meant to help us test the training loop, allowing for quick training time and almost immediate feedback on changes. It is comprised of a multi layer perceptron with 2 hidden layers of sizes 120 and 84. The input is given by the observation space of the environment. The output size is $|\mathcal{A}| \times N$. ReLU activations are used after each fully connected layer except for the last one.

The second implementation is tailored to the Atari 2600 tasks. To enable direct comparison with previously reported results, we construct the same convolutional neural network as Dabney et al. (2017), which is adapted from Mnih et al. (2015) as explained above. Concretely, on the processed $4 \times 84 \times 84$ dimensional observations, the first layer applies a 32 filter convolution of size 8 and stride 4. The second convolutional layer uses 64 filters, size of 4 and stride of 4. The third and final convolution uses the same number of filters, a size of 3 and stride of 1. The result is flattened and taken through fully connected layer with 512 dimensions. The output size is the same as above. Again, ReLU activations are used after each layer except the last one.

3.2. Environment Setup

Reinforcement learning performance is particularly sensitive to even minor environment or hyperparameter changes. This is particu-

larly true for ALE, which is why replicating the exact settings used in the literature is of utmost importance to ensure the validity of our results.

Gymnasium provides so called wrappers meant to modify the behavior of the environment’s `step` and `init` methods in generally useful ways. Because of certain particularities of the Atari games the exact sequence of wrappers applied for this task has become standard. In the following we summarize the reasoning behind the most important ones for RL training as explained by Mnih et al. (2015). In the order they are applied:

1. **NoopResetEnv**: Starts each episode with n noops (no operation) in order to randomize the starting state. In general, $n = 30$.
2. **MaxAndSkipEnv**: Whenever we call `step` increment k frames and output the maximum of the last two (of the k skipped). The agent’s last action is repeated on skipped frames. Frame skipping enables the agent to play approximately k times more games within the same runtime, since stepping the emulator is much faster than computing the next move (Mnih et al., 2015). The maximum over the last two frames is taken in order to prevent flickering. Due to hardware constraints some Atari games would draw sprites on alternative frames. $k=4$ in general.
3. **EpisodicLifeEnv**: For games with a life counter, ends an episode when a life is lost. However this does not affect episode reset, which only happens on true game over.
4. **FireResetEnv**: Some environments (such as Pong) require a ‘Fire’ action in order to start. This wrapper performs the action on behalf of the agent if needed.
5. **ResizeObservation**: The raw 210x160 pixel frames from Atari are rescaled to 84x84. This is performed for memory optimization reasons.
6. **GrayScaleObservation**: Extract only the luminance channel from the observations.
7. **FrameStack**: Stack the last m frames in a new dimension. This gives the agent information about direction of motion of sprites, which would otherwise be unavailable. Typically $m=4$.

Note that out of the above, Gymnasium only provides **ResizeObservation**, **GrayScaleObservation** and **FrameStack**, the rest being custom CleanRL [implementations](#).

Thankfully CleanRL provides algorithm implementations specific to Atari which contain the above setup and should typically not be modified. The same is true for our implementation. (see Figure 6)

3.3. Training loop and loss function

In this chapter we will focus on the main sequence of operations required in the training loop as well as some particular implementation details. The training loop simultaneously accomplishes two main tasks: data gathering by stepping through the environment and updating of weights.

The exact sequence of events is as follows. At initialization, we keep a global environment with which our agent can interact. At each step, given the current observation from the environment, the ϵ -greedy policy picks an action uniformly at random with probability ϵ or, alternatively, the action with the highest predicted Q -value. This is used to step forward the environment, getting the next observation and potentially a reward. The tuple of initial observation, action, reward, next observation is saved in a so called replay buffer for sampling during training.

Table 1: Best agent performance on the Atari-5 games.

Game	DQN	C51	QR-DQN-0	QR-DQN-1	Ours
Battle Zone	29,900	28,742	35,580	39,268	49,500
Double Dunk	-6.6	2.5	12.3	21.9	18.8
Name this Game	8,207.8	12,542	17,557	21,890	23,013
Phoenix	8,485.2	17,490	65,767	16,585	124,298
Q*Bert	13,117.3	23,784	26,946	572,510	26,822

Next, we aim to update the weights of our training model. For this, we uniformly sample a batch of action value pairs from the replay buffer. The quantile distributions of the current and next action are calculated under the training model respectively the target model. We then follow Algorithm 1 to compute the Bellman operator and the quantile Huber loss. The gradients of this loss with respect to our target network can be computed and weights updated using SGD.

Finally, depending on the current step, the target network may be synchronized with the training network.

Ensuring stability

As stated in Section 2.5, for stability reasons it is preferable to use a separate target network which remains fixed while gradient descent is being performed on the training network.

A replay buffer is used to store and sample from past state action pairs and their outcomes. The advantage of this is twofold. Firstly, it allows us to batch training, thus leading to less noisy gradients. Secondly, random sampling breaks the time correlation within sequential samples. If we were to perform a weigh update based only on the last seen states, our model would overfit to that set of very similar experiences. Moreover, since those experiences are sampled from the model we are training, their distribution is shifting, again introducing stability issues for SGD.

Moreover, the exploration probability ϵ is linearly interpolated between two values for the first steps, and then is fixed for the remainder of training. Intuitively, this allows for more random exploration when the model hasn't yet had time to learn anything meaningful, thus supplying it with a diverse set of experiences. On the other hand, later in training, high ϵ can cause the agent to make blunders where the non random policy would have taken the right action. Thus it would be highly likely that the agent would miss out on reaching high return states, sabotaging its learning progress.

These methods have become standard in reinforcement learning thanks to Mnih et al. (2015).

CleanRL implementation

We adapted the training loop from the C51 [implementation](#) in CleanRL. Our main contribution was the efficient computation of the Bellman operator and the quantile Huber loss mirroring Algorithm 1. Testing of our code is outlined in Section 4

4. Experimental Results

We empirically verified the correctness of our implementations by training on 2 tasks: Classic Control, Atari 2600. The first enabled us to run quick iterations for eliminating initial bugs within the training loop, while thanks to the Atari 2600 games we were able to directly compare our performance with that of Bellemare et al. (2017).

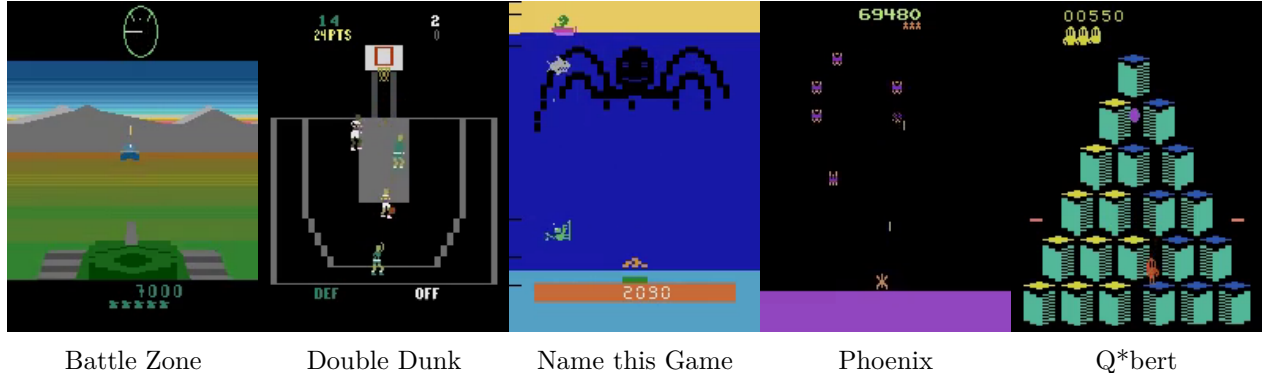


Figure 3: Screenshots of the Atari-5 games

All models were trained on A100 GPUs made available by École Polytechnique. The particular hyperparameters and training time are specified below for each task.

As is standard in CleanRL, for each training session we log the Q-value, the quantile Huber loss, episodic return and episode length. Episodic return is of particular interest as it gives us an indication for how the model is learning. This signal is however very noisy due to random exploration. Especially in certain Atari games we looked at (Qbert is one example) these random moves can be enough to massively influence the score by making the agent lose multiple lives. For this reason, when training on Atari games, we found it useful to additionally log a test score averaged over 3 validation episodes with $\epsilon = 0.001$. This was only logged every 1 million steps for performance reasons.

4.1. Classic Control

We started by testing our initial implementation on the classic control environments provided by Gymnasium. For this task, CleanRL Q-learning models typically benchmark on the CartPole-v1, Acrobot-v1 and MountainCar-v0 environments, we used the same. For each task, we used CleanRL’s benchmarking utility to train the model for 3 seeds, with the validation hyperparameters as in Table 2. These are standardized

by CleanRL for easy comparison across implementations. The averaged episodic return curves may be seen in Figure 4. As expected, QR-DQN reliably learns to master these tasks allowing us to proceed with additional testing.

4.2. Atari 2600

The Arcade Learning Environment (ALE) (Bellemare et al., 2013) has become a classic task for reinforcement learning algorithms. It provides a training interface for a set of 57 Atari 2600 games. ALE provides a challenging task for both humans and machines, along with standardized methodology allowing for direct comparisons. Many works report their results on this benchmark, most importantly Bellemare et al. (2017) and Dabney et al. (2017) for C51 and QR-DQN respectively. As such, performance on ALE is the obvious benchmark choice to validate our implementation.

ALE provides a standard interface for all games: observations are represented by the raw 210×160 pixel rgb values; actions are discrete with a maximum of 18 possible values of which 5 controller inputs, 12 button combinations and 1 noop (many games use only a subset of the actions); rewards are specific to each game, usually given by the score.

Due to restrictions imposed by compute

and training time, we limit ourselves to the Atari-5 subset (Aitchison et al., 2022) composed of Battle Zone, Double Dunk, Name this Game, Phoenix, and Q*bert (see Figure 3). Aitchison et al. (2022) show that scores on these games are representative of performance on the entire dataset.

The environment settings used can be found in Table 3, while the preprocessing done to the observations are outlined in Section 3.2. It needs to be mentioned that ALE provides multiple variants of each environment with different settings. CleanRL, and by extension our implementation, use the `NoFrameskip-v4` variants. This allows us to set frame skip through environment wrappers. We also keep repeat action probability to 0.

Training on each game was performed with the model outlined in 3.1 and hyperparameters from (Dabney et al., 2017) outlined in Table 3. Complete training took around 24 hours for each environment.

One possible point of confusion when transposing the paper hyperparameters to CleanRL is the relationship of frames to steps. While Bellemare et al. (2017) report training from 200 million frames, in CleanRL this corresponds to 50 million steps because of the frame skip of 4. Furthermore, note that CleanRL only requires runs of 10 million steps, as opposed to 50 million. This is because they usually report performance on Beam Rider, Breakout and Pong which are generally easier to learn. Nevertheless, we trained for 50 million steps in our experiments in order to directly compare our results to Dabney et al. (2017).

For the Adam optimizer, we set the epsilon parameter to $\frac{0.01}{\text{batch size}}$. For a batch size of 32, this is equivalent to Dabney et al. (2017). Replay buffer size, target network frequency, train frequency come from the values set for other algorithms in CleanRL. All the other parameters are taken from Dabney et al.

(2017).

We report results under two evaluation regimes:

- **Best agent performance:** Every one million steps, training is paused and we evaluate our agent for 10 episodes with $\epsilon = 0.001$, taking the mean of the scores. We keep track of the highest mean score archived. Note that Bellemare et al. (2017) evaluate for 500k steps instead of 10 episodes. The results for this training regime can be found in Table 1.
- **Online Performance:** We provide the episodic return during training (training curves) averaged over 3 seeds and smoothed over a sliding window of 5 iterations. See results in Figure 5.

Looking at Table 1, our implementation runs as expected, outperforming Dabney et al. (2017)’s implementation on 3 of the 5 Atari games. We conclude it is robust and ready to submit to CleanRL.

5. Conclusion

In this paper we have built a QR-DQN implementation from the ground up, linking each part of the code to the theory outlined by Dabney et al. (2017). We have clearly pointed out and motivated the RL standard practices introduced by Mnih et al. (2015). Finally we have shown how our implementation was tested and we empirically proved its robustness when compared against previous works. This implementation has been submitted to the CleanRL repository and it is on track to being fully integrated into it.

References

- Matthew Aitchison, Penny Sweetser, and Marcus Hutter. Atari-5: Distilling the arcade learning environment down to five games, 2022. URL <https://arxiv.org/abs/2210.02019>.
- M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. ISSN 1076-9757. doi: 10.1613/jair.3912. URL <http://dx.doi.org/10.1613/jair.3912>.
- Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning, 2017. URL <https://arxiv.org/abs/1707.06887>.
- Richard Bellman. Dynamic programming. *Princeton University Press, Princeton, NJ, USA*, 1:3–25, 1958.
- Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression, 2017. URL <https://arxiv.org/abs/1710.10044>.
- Peter J. Huber. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964. doi: 10.1214/aoms/1177703732. URL <https://doi.org/10.1214/aoms/1177703732>.
- Roger Koenker. *Quantile Regression*. Number 38 in Econometric Society Monographs. Cambridge University Press, Cambridge, 2005. doi: 10.1017/CBO9780511754098. URL <https://doi.org/10.1017/CBO9780511754098>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236. URL <https://doi.org/10.1038/nature14236>.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: an introduction, 2nd edn. adaptive computation and machine learning*. The MIT Press, Cambridge, 2018.

A. Hyperparameters

Descriptions

- **Total timesteps:** Total environment steps to run the training for.
- **Learning rate:** Learning rate parameter for the ADAM optimizer.
- κ : parameter for the Huber loss. Setting to 0 makes the loss behave like simple quantile regression.
- **Replay buffer size:** number of state action pairs to be kept in the replay buffer to sample from for weight updates.
- γ : Discount factor used in the TD update.
- **Target network frequency:** How many steps to wait between synchronizations of the train and target networks.
- **Batch size:** Batch size for gradient updates. Also influences the epsilon parameter of ADAM which is set to $\frac{0.01}{\text{Batch size}}$.
- **Start ϵ :** Starting value for the linear interpolation of the exploration probability.
- **End ϵ :** Final value for the linear interpolation of the exploration probability.
- **Exploration fraction:** Defines the duration of the interpolation of ϵ as $\text{Exploration fraction} \times \text{Total timesteps}$.
- **Learning start:** Step after which to start performing gradient updates. This is done to give the replay buffer time to fill up with varied experiences.
- **Train frequency:** How many steps to wait between weight updates.

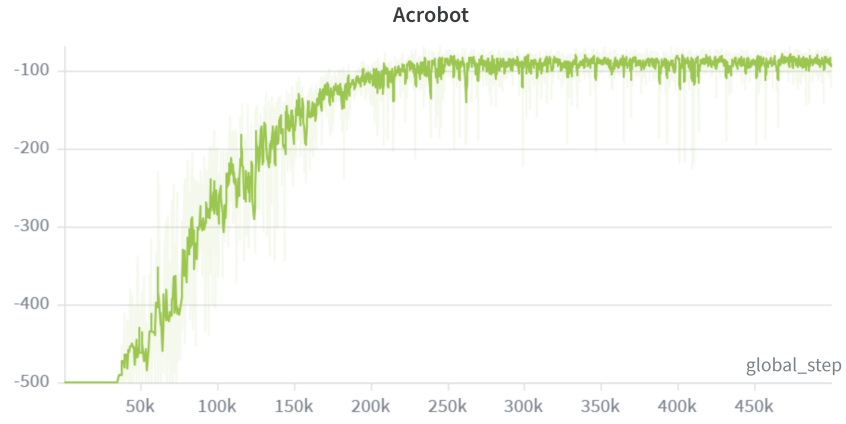
Table 2: Complete Hyperparameters for training on Classic Control.

Parameter	Value
Total timesteps	500,000
Learning rate	2.5e-4
N	200
κ	1
Replay buffer size	10,000
γ	0.99
Target network frequency	500
Batch size	128
Start ϵ	1
End ϵ	0.05
Exploration fraction	0.5
Learning start	10,000
Train frequency	10

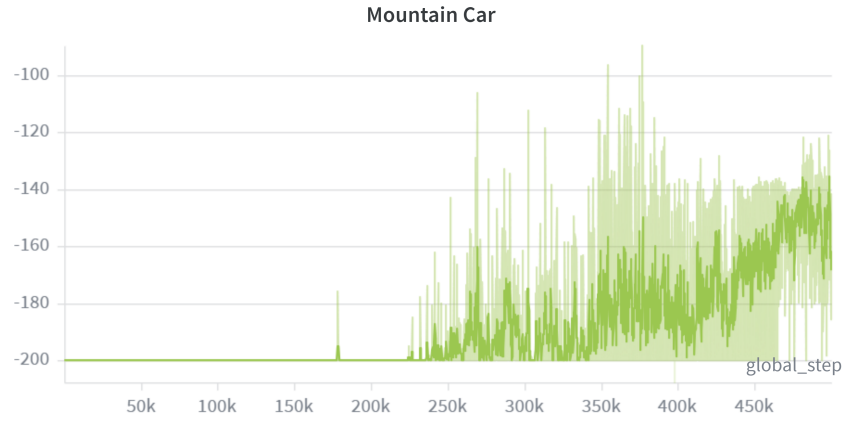
Table 3: Complete Hyperparameters for training on the Atari 2600 games.

Parameter	Value
Total timesteps	50M
Learning rate	5e-5
N	200
κ	1
Replay buffer size	1M
γ	0.99
Target network frequency	10,000
Batch size	32
Start ϵ	1
End ϵ	0.01
Exploration fraction	0.005
Learning start	50,000
Train frequency	4

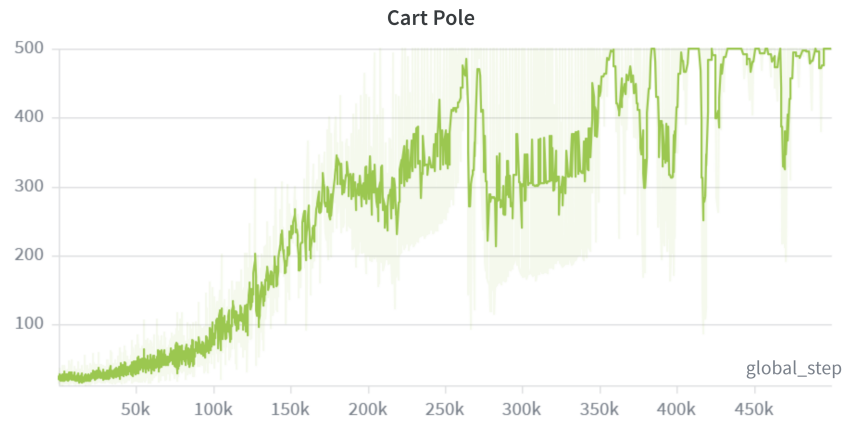
B. Training Curves



(a) Acrobot

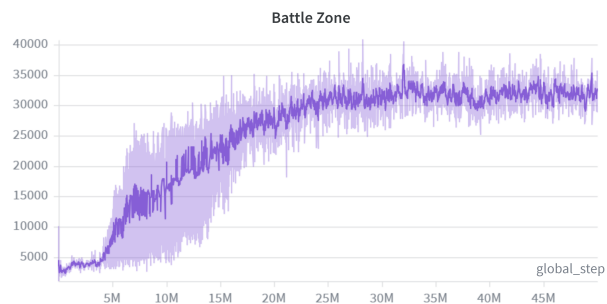


(b) Mountain Car

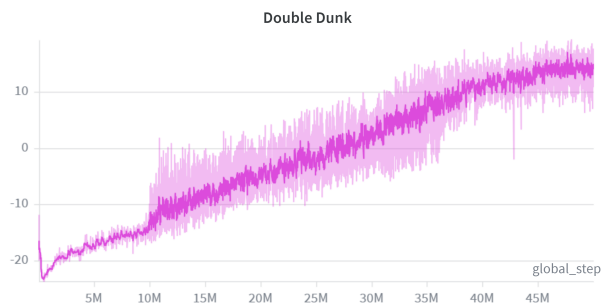


(c) Cart Pole

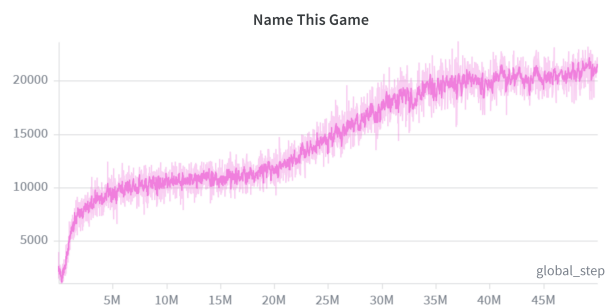
Figure 4: Episodic return averaged over 3 seeds and a sliding window of 5 for Classic Control tasks. Shaded area represents standard deviation.



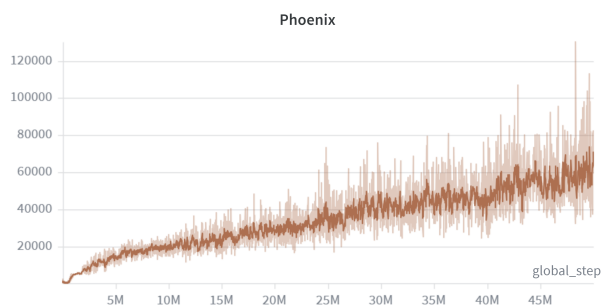
(a) Battle Zone



(b) Double Dunk



(c) Name this Game



(d) Phoenix



(e) Q*bert

Figure 5: Evaluation Atari-5 games under the online performance regime.

C. Code

```
1 def make_env(env_id, seed, idx, capture_video, run_name):
2     def thunk():
3         if capture_video and idx == 0:
4             env = gym.make(env_id, render_mode="rgb_array")
5             env = gym.wrappers.RecordVideo(env, f"videos/{run_name}")
6         else:
7             env = gym.make(env_id)
8             env = gym.wrappers.RecordEpisodeStatistics(env)
9
10            env = NoopResetEnv(env, noop_max=30)
11            env = MaxAndSkipEnv(env, skip=4)
12            env = EpisodicLifeEnv(env)
13            if "FIRE" in env.unwrapped.get_action_meanings():
14                env = FireResetEnv(env)
15            env = ClipRewardEnv(env)
16            env = gym.wrappers.ResizeObservation(env, (84, 84))
17            env = gym.wrappers.GrayScaleObservation(env)
18            env = gym.wrappers.FrameStack(env, 4)
19
20            env.action_space.seed(seed)
21            return env
22
23     return thunk
```

Figure 6: CleanRL environment generator function showcasing the use of the wrappers mentioned in 3.2. Code taken from [here](#)

```

1 # ALGO LOGIC: training.
2 if global_step > args.learning_starts:
3     if global_step % args.train_frequency == 0:
4         data = rb.sample(args.batch_size)
5         with torch.no_grad():
6             _, target_quantiles = target_network.get_action(data.
7                 next_observations)
8             target_quantiles = data.rewards + args.gamma *
9                 target_quantiles * (1 - data.dones)
10
11             _, pred_quantiles = q_network.get_action(data.observations,
12                 data.actions.flatten())
13
14             quantile_diff = target_quantiles.unsqueeze(1) -
15                 pred_quantiles.unsqueeze(2) # (batch_size, n_quantiles,
16                 n_quantiles)
17             # dim 1 = i = quantiles of the old action
18             # dim 2 = j = quantiles of the next action
19
20             huber_loss = torch.nn.functional.huber_loss(quantile_diff,
21                 torch.zeros_like(quantile_diff), reduction="none", delta=
22                 args.k_huber)
23             quantile_loss = torch.abs(q_network.tau_hat[:, None] - (
24                 quantile_diff.detach() < 0).float()) * huber_loss
25
26             # expectation over j, sum over i, then batch
27             loss = quantile_loss.sum(1).mean()
28
29             if global_step % 100 == 0:
30                 writer.add_scalar("losses/loss", loss.item(),
31                     global_step)
32                 pred_val = pred_quantiles.mean(dim=1) # (batch_size,)
33                 writer.add_scalar("losses/q_values", pred_val.mean().
34                     item(), global_step)
35                 print("SPS:", int(global_step / (time.time() -
36                     start_time)))
37                 writer.add_scalar("charts/SPS", int(global_step / (time.
38                     time() - start_time)), global_step)
39
40             # optimize the model
41             optimizer.zero_grad()
42             loss.backward()
43             optimizer.step()

```

Figure 7: Training step code from our QR-DQN implementation. With the exception of the weight updates at the end, this is exactly equivalent to Algorithm 1. Full code [here](#)